

Testergebnisse des ICD-Suchsystems

3 May 2026

Inhaltsverzeichnis

1	Testergebnisse – Person A: Grundlagen und Datenqualität	2
1.1	Ziel des Tests	2
2	Testergebnisse – Person B: Suchpipeline und Fallback-Logik	4
2.1	Ziel des Tests	4
3	Testergebnisse – Person C: LLM, Cache und Systemverhalten	6
3.1	Ziel des Tests	6

1 Testergebnisse – Person A: Grundlagen und Datenqualität

1.1 Ziel des Tests

Ziel der Tests war die Sicherstellung der korrekten Datenbasis sowie der grundlegenden Systemfunktionen. Dabei wurde geprüft, ob ICD-Codes zuverlässig erkannt, Synonyme korrekt zugeordnet und Datenbankeinträge konsistent verarbeitet werden.

Durchgeführte Tests

1. Unit Tests

ICD Code Detection

Was wurde getestet:

Erkennung von ICD-Codes sowie medizinischen Begriffen.

Wie wurde getestet:

Es wurden verschiedene Eingaben an die Suchlogik übergeben und überprüft, ob das System die korrekten ICD-Codes erkennt.

Input	Erwartetes Verhalten	Ergebnis
E11.9	ICD-Code wird erkannt	Bestanden
diabetes	Medizinischer Begriff wird erkannt	Bestanden
I21	ICD-Code wird erkannt	Bestanden

Synonym Lookup

Was wurde getestet:

Zuordnung von medizinischen Synonymen zu den korrekten ICD-Codes.

Wie wurde getestet:

Mehrere Synonyme wurden in die Datenbankabfrage eingegeben und mit den erwarteten ICD-Codes verglichen.

Synonym	Erwarteter ICD-Code	Ergebnis
Altersdiabetes	E11.9	Bestanden
Herzinfarkt	I21	Bestanden
Lumboischialgie	M54.4	Bestanden

Search Routing

Was wurde getestet:

Korrekte Weiterleitung von Suchanfragen an die jeweilige Suchlogik.

Wie wurde getestet:

Unterschiedliche Suchtypen wurden eingegeben und geprüft, ob die passende Suchstrategie verwendet wurde.

Ergebnis:

Alle Suchanfragen wurden korrekt verarbeitet.

LLM Prompt Erstellung**Was wurde getestet:**

Erstellung konsistenter und strukturierter Prompts für das LLM.

Wie wurde getestet:

Automatisch generierte Prompts wurden analysiert und auf Vollständigkeit überprüft.

Ergebnis:

Prompts wurden korrekt erstellt.

2. Datenbank Tests**Datenbank Integrität****Was wurde getestet:**

Überprüfung der Datenbank auf ungültige Referenzen.

Wie wurde getestet:

Mittels SQL-Abfrage wurden mögliche fehlerhafte Synonym-Zuordnungen gesucht.

SQL-Test:

```
SELECT COUNT(*) FROM invalid_mappings;
```

Konkretes Resultat:

- Gefundene ungültige Einträge: 0
- Ergebnis: Bestanden

Zusammenfassung der Resultate

Kategorie	Anzahl
Durchgeführte Tests	10
Erfolgreiche Tests	10
Fehlgeschlagene Tests	0

Gesamtbewertung:

Die grundlegenden Systemfunktionen sowie die Datenbankintegrität funktionieren stabil und fehlerfrei. Alle definierten Tests konnten erfolgreich abgeschlossen werden.

2 Testergebnisse – Person B: Suchpipeline und Fallback-Logik

2.1 Ziel des Tests

Ziel der Tests war die Überprüfung der mehrstufigen Suchpipeline inklusive der Fallback-Strategien zwischen Meilisearch, pgvector und LLM-Korrektur.

Durchgeführte Tests

1. Pipeline Tests

Suchpipeline

Was wurde getestet:

Korrekte Auswahl und Nutzung der verschiedenen Suchstufen.

Wie wurde getestet:

Mehrere Suchbegriffe mit unterschiedlicher Qualität wurden eingegeben. Anschließend wurde geprüft, welche Suchstufe verwendet wurde und ob ein korrektes Ergebnis zurückgegeben wurde.

Input	Verwendete Stufe	Ergebnis
E11.9	Meilisearch	Bestanden
Altersdiabetes	Meilisearch	Bestanden
Zuckerkrankheit	pgvector	Bestanden
diabtes typ 2	LLM-Korrektur	Bestanden
xyzabc	Kein Treffer	Bestanden

Detailanalyse: Zuckerkrankheit

Was wurde getestet:

Fallback von Meilisearch auf semantische Suche.

Wie wurde getestet:

Der Begriff wurde eingegeben und der Ablauf der Suchpipeline analysiert.

Konkretes Resultat:

- Meilisearch: Kein Treffer
- pgvector: Treffer (E11.9)
- Ergebnis korrekt erkannt

Detailanalyse: diabtes typ 2

Was wurde getestet:

Automatische Korrektur fehlerhafter Eingaben mittels LLM.

Wie wurde getestet:

Ein absichtlich falsch geschriebener Begriff wurde eingegeben und der Suchprozess beobachtet.

Konkretes Resultat:

- Meilisearch: Kein Treffer
- pgvector: Kein Treffer
- LLM-Korrektur: „Diabetes Typ 2“
- Erneute Suche: Treffer (E11.9)
- Ergebnis korrekt erkannt

2. Fallback Validierung

Was wurde getestet:

Korrekte Aktivierung der einzelnen Fallback-Stufen.

Wie wurde getestet:

Mehrere Suchanfragen ohne direkten Treffer wurden durchgeführt.

Konkretes Resultat:

- Reihenfolge korrekt eingehalten
- Keine unnötigen Fallbacks beobachtet

Zusammenfassung der Resultate

Kategorie	Anzahl
Durchgeführte Tests	12
Erfolgreiche Tests	11
Fehlgeschlagene Tests	1

Gesamtbewertung:

Die Suchpipeline funktioniert zuverlässig. Besonders die Fallback-Mechanismen und die LLM-Korrektur verbessern die Robustheit der Suche deutlich.

3 Testergebnisse – Person C: LLM, Cache und Systemverhalten

3.1 Ziel des Tests

Ziel war die Überprüfung der LLM-Integration, der Cache-Funktion sowie des Systemverhaltens unter realistischen Bedingungen.

Begriffe

- **k6**: Tool für Lasttests (sendet viele parallele Anfragen).
- **VU (Virtual User)**: simulierter Benutzer im Lasttest.
- **p95**: 95 Prozent der Anfragen waren schneller als dieser Wert.

Durchgeführte Tests

1. Cache Tests

Was wurde getestet

Korrekte Nutzung des Caches zur Vermeidung unnötiger API-Aufrufe sowie die Verbesserung der Antwortzeiten.

Wie wurde getestet

Jede Diagnose wurde zweimal abgefragt:

- erster Aufruf ohne Cache
- zweiter Aufruf mit Cache

Zusätzlich wurde geprüft, ob der Cache-Schlüssel korrekt aufgebaut ist, damit verschiedene Antworttypen getrennt gespeichert werden.

Konkrete Resultate

Aufruf	Verhalten	Antwortzeit
1. Aufruf	LLM wird verwendet	3–7 Sekunden
2. Aufruf	Cache wird verwendet	unter 60 ms

Das System war dadurch etwa 60- bis 100-mal schneller.

Der Cache-Schlüssel besteht aus:

- ICD-Code
- Endpunkt-Typ

Dadurch werden beispielsweise Facharzt-Antworten und Erklärungs-Antworten nicht vermischt.

2. Verhalten unter Last

Was wurde getestet

Stabilität und Performance des Systems bei mehreren gleichzeitigen Benutzern.

Wie wurde getestet

Die Lasttests wurden mit **k6** durchgeführt. Dabei wurden mehrere virtuelle Benutzer (VU) simuliert.

Es wurden zwei Szenarien getestet:

- Best-Case mit warmem Cache
- Worst-Case mit kaltem Cache

Best-Case: Warmer Cache

Testaufbau

- 20 parallele Benutzer
- Dauer: 60 Sekunden
- gleiche Anfrage

Konkrete Resultate

- 11'254 Anfragen
- 0 Fehler
- $p_{95} = 20$ ms
- 187 Anfragen pro Sekunde

Die Zielwerte wurden klar erfüllt:

- Fehlerrate < 1 %
- $p_{95} < 200$ ms

Worst-Case: Kalter Cache

Testaufbau

- 5 parallele Benutzer
- 10 geleerte Diagnosen
- Dauer: 60 Sekunden

Konkrete Resultate

- 0 Fehler

- Median: 4 ms
- p95 = 3.7 Sekunden

Die höheren p95-Werte entstanden ausschliesslich durch die ersten LLM-Aufrufe. Während des Tests füllte sich der Cache automatisch.

3. Kontext im Chat

Was wurde getestet

Ob der Endpunkt `/api/chat/contextual` die aktuelle Diagnose korrekt als Kontext verwendet.

Wie wurde getestet

Dieselbe Frage wurde mit unterschiedlichen Diagnosen gestellt:

„Darf ich Sport machen?“

Anschliessend wurden die Antworten verglichen.

Konkrete Resultate

Diagnose: Diabetes Typ 2

- Antwort bezog sich auf:
 - Blutzucker
 - Insulin
 - Hypoglykämie

Diagnose: Hypertonie

- Antwort bezog sich auf:
 - Blutdruck
 - Gefässe
 - Belastungs-EKG

Die Antworten waren krankheitslogisch passend. Der gesetzte Kontext beeinflusste die Antwort korrekt.

4. GUI und Usability Tests

Was wurde getestet

Benutzerfreundlichkeit und Verständlichkeit der Oberfläche.

Wie wurde getestet

Mehrere Testpersonen führten typische Suchaufgaben innerhalb der Anwendung durch.

Konkrete Resultate

Positiv:

- intuitive Bedienung
- verständliche Suchergebnisse
- schnelle Reaktionszeiten

Festgestellte Verbesserungsmöglichkeiten:

- nicht-klickbarer Block wirkte klickbar
- zu harter Fehlerpfad in der Suche

Kernaussagen

- Mit warmem Cache lagen 95 Prozent aller Antworten unter 20 ms.
- Das System blieb bei 20 parallelen Benutzern vollständig stabil.
- Mit kaltem Cache blieb das System ebenfalls stabil und der Cache füllte sich selbstständig.
- Der Kontext-Chat funktionierte nachweislich korrekt.
- Der Usability-Test zeigte zwei konkrete Verbesserungsmöglichkeiten in der Benutzeroberfläche.

Gesamtbewertung

Die LLM-Integration, Cache-Mechanismen sowie das allgemeine Systemverhalten funktionieren zuverlässig. Besonders die Cache-Logik verbessert die Performance deutlich. Auch unter Last blieb das System stabil und benutzerfreundlich.